$$x_{new} = x_{old} - \alpha \frac{df}{dx}$$

Let’s see how to do this in code.

Listing A.1. Gradient descent

```
import numpy as np

def f(x):                          1
    return np.log(np.power(x,4) + np.power(x,3) + 2)

def dfdx(x):                       2
    return (4*np.power(x,3) + 3*np.power(x,2)) / f(x)

x = -9.41                          3
lr = 0.001                         4
epochs = 5000                      5
for i in range(epochs):
    deriv = dfdx(x)                6
    x = x - lr * deriv             7
```

- ***1*** The original function
- ***2*** The derivative function
- ***3*** Random starting point
- ***4*** Learning rate (step size)
- ***5*** Number of iterations to optimize over
- ***6*** Calculates derivative of current point
- ***7*** Updates current point

If you run this gradient descent algorithm, you should get $x = -0.750000000882165$, which is (if rounded) exactly what you get when calculated algebraically. This simple process is the same one we use when training deep neural networks, except that deep neural networks are multivariable compositional functions, so we use partial derivatives. A partial derivative is no more complex than a normal derivative.

Consider the multivariable function $f(x,y) = x^4 + y^2$. There is no longer a single derivative of this function since it has two input variables. We can take the derivative with respect to x or y or both. When we take the derivative of a multivariable function with respect to all of its inputs and